BUSCA E ORDENAÇÃO · PROJETO 1

Buscador de filmes e cinemas

Índices, árvore e buscas medidas em 584 mil filmes

Módulo 1 · tipo e gênero **Módulo 2 · ano e duração**

Prof. Tadeu Zubaran · Vitor Tavares Simões, Michael Douglas Teixeira Barbosa · 20/10/2026

Responder buscas em 584 mil filmes sem olhar todos eles

584.121

filmes

48 MB de texto, uma linha por filme, colunas no estilo IMDb

400

cinemas

cada um com sua lista de filmes em cartaz: 2.009 no total

5

tipos de busca

tipo e gênero com E / OU, duração, ano e cinemas

O enunciado aceita um carregamento demorado, desde que a consulta seja rápida. Por isso o programa **prepara tudo ao abrir** e cada busca só usa o que já está pronto.

Preparar uma vez, responder muitas

PREPARAR · UMA VEZ

0,75 s

ler os dois arquivos e montar índices e árvores, ao abrir o programa



RESPONDER · CADA BUSCA

0,03 ms

média de 703 buscas, usando só o que foi preparado

Ideia central: cada filme existe uma vez só na memória. Índices e árvores guardam apenas a posição dele no vetor, um número inteiro.

`filmes =` `[0] The Battle II` `[1] Miguel jura…` `[2] El amor…` … `índice "short" = [0, 3]`

Como o código está organizado

Arquivo	Responsabilidade
modelos.h	struct Filme e struct Cinema
texto.cpp	dividir(), aparar(), converterInteiro()
entrada.cpp	lerLinha(), lerInteiro(): entrada validada
carregador.cpp	lê os dois arquivos e liga cinema ⟶ filme
indice.cpp	IndiceCategoria, intersecao(), uniao() · Módulo 1
arvore.cpp	ArvoreNumerica: baldes, montar, coletar · Módulo 2
cache.cpp	CacheConsultas: chave ⟶ resultado
main.cpp	monta tudo, menu e as 5 buscas

main(), na ordem em que roda

```
1 carregarFilmes() → vector<Filme>
2 carregarCinemas() → vector<Cinema>
3 resolverFilmesDosCinemas()
4 laço: 2 índices + 2 árvores
5 arvoreAno / arvoreDuracao.montar()
6 menu → uma das 5 buscas
```

Uma linha do arquivo vira um Filme

src/carregador.cpp · carregarFilmes() · trechos das linhas 17–50

```
17  while (std::getline(arquivo, linha)) {
22      std::vector<std::string> campos = dividir(linha, '\t');
23      if (campos.size() < 9) {
24          linhasIgnoradas++;
25          continue;
26      }
27      campos[8] = aparar(campos[8]);
 :    copia id, tipo e título · ano e duração começam em -1
36      bool anoValido = (campos[5] == "\\N") || converterInteiro(campos[5], filme.ano);
37      bool duracaoValida = (campos[7] == "\\N") || converterInteiro(campos[7], filme.duracao);
 :    linha inválida é pulada · gêneros divididos pela vírgula
50      filmes.push_back(filme);
```

L17 uma linha por vez: memória de uma linha, não dos 48 MB

L36–37 \N vira −1; texto que não é número pula a linha

L50 a posição no vetor vira o “id” de todas as estruturas

Um único laço monta os índices e as árvores

src/main.cpp · main() · linhas 339–353

```
339  for (size_t i = 0; i < filmes.size(); i++) {
340      int posicao = static_cast<int>(i);
341      indiceTipo.adicionar(filmes[i].tipo, posicao);
342      for (const std::string& genero : filmes[i].generos) {
343          indiceGenero.adicionar(genero, posicao);
344      }
345      if (filmes[i].ano >= 0) {
346          arvoreAno.adicionar(filmes[i].ano, posicao);
347      }
348      if (filmes[i].duracao >= 0) {
349          arvoreDuracao.adicionar(filmes[i].duracao, posicao);
350      }
351  }
352  arvoreAno.montar();
353  arvoreDuracao.montar();
```

L339 uma passada pelos 584.121 filmes; **posicao** é o que todas as estruturas guardam

> **L341–344** Módulo 1: o filme entra na lista do seu tipo e na de cada gênero

> **L345–350** Módulo 2: ano e duração vão para os baldes; −1 (sem informação) fica de fora

> **L352–353** só depois de tudo lido: liga os baldes em árvore, uma vez

Índices: 44,2 ms · baldes: 7,5 ms · montar: 0,08 ms (tests/logs/benchmark.tsv)

MÓDULO 1 · ÍNDICE POR CATEGORIA

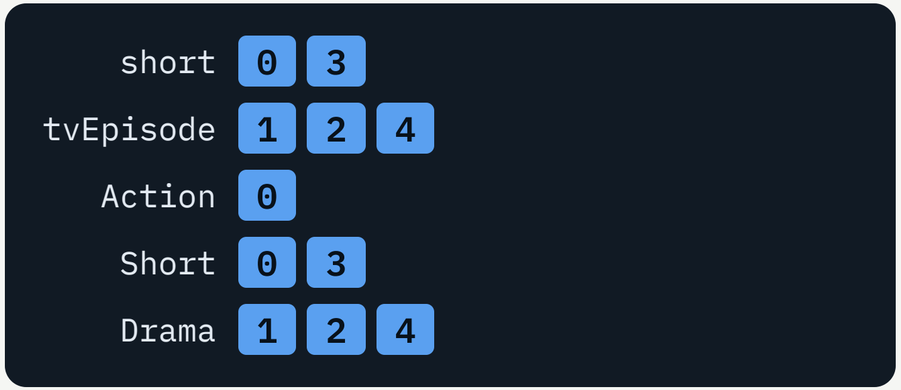
IndiceCategoria: uma lista de posições por nome

src/indice.cpp · IndiceCategoria::adicionar() · linhas 3–12

```
3   void IndiceCategoria::adicionar(const std::string& chave,
                                    int indiceFilme) {
4       for (size_t i = 0; i < chaves.size(); i++) {
5           if (chaves[i] == chave) {
6               listas[i].push_back(indiceFilme);
7               return;
8           }
9       }
10      chaves.push_back(chave);
11      listas.push_back(std::vector<int>{indiceFilme});
12  }
```

L4–7 nome já existe: a posição vai para o fim da lista dele

L10–11 nome novo: cria a lista já com o filme; chaves[i] ↔ listas[i]



Os 5 primeiros filmes do arquivo. O filme 0 é Action,Short e entra em duas listas.

Posições entram na ordem 0, 1, 2...: **toda lista já sai crescente**. A seção do E / OU depende disso.

MÓDULO 1 · COMBINAR FILTROS

aplicarFiltros: combina as listas, filtro a filtro

src/main.cpp · aplicarFiltros() · linhas 108–121

```
108  std::vector<int> resultado;
109  for (size_t i = 0; i < filtros.size(); i++) {
110      const IndiceCategoria& indice = (filtros[i].tipoCampo == "tipo") ? indiceTipo : indiceGenero;
111      std::vector<int> parcial = indice.buscar(filtros[i].valor);
113      if (i == 0) {
114          resultado = parcial;
115      } else if (operador == "OU") {
116          resultado = uniao(resultado, parcial);
117      } else {
118          resultado = intersecao(resultado, parcial);
119      }
120  }
121  return resultado;
```

L110–111 cada filtro escolhe o índice (tipo ou gênero) e pega a lista pronta

L116 / L118 combina com o resultado até aqui: OU = união, E = interseção

Como as duas devolvem listas **em ordem**, dá para encadear 3 ou mais filtros

Exemplo: tipo=movie E genero=Comedy → intersecao(lista de movie, lista de Comedy) → 3.382 filmes

E / OU: as duas listas andam juntas

src/indice.cpp · intersecao()

```
27  while (i < a.size() && j < b.size()) {
28      if (a[i] == b[j]) {
29          resultado.push_back(a[i]);
30          i++;
31          j++;
32      } else if (a[i] < b[j]) {
33          i++;
34      } else {
35          j++;
36      }
37  }
```

a = [0, 3, 5, 9] · b = [3, 4, 9]

a[i]	b[j]	decisão
0	3	0 < 3 → avança i
3	3	iguais → 3 entra
5	4	4 < 5 → avança j
5	9	5 < 9 → avança i
9	9	iguais → 9 entra

As listas estão em ordem. Se **a[i] < b[j]**, a[i] nunca vai aparecer em b: descarta sem procurar. No máximo **|a| + |b|** passos. O OU segue a mesma ideia.

MÓDULO 1 · FILTRO OU

uniao: a cada passo entra o menor dos dois

src/indice.cpp · uniao() · linhas 45–65

```
45  while (i < a.size() && j < b.size()) {
46      if (a[i] == b[j]) {
47          resultado.push_back(a[i]);
48          i++;
49          j++;
50      } else if (a[i] < b[j]) {
51          resultado.push_back(a[i]);
52          i++;
53      } else {
54          resultado.push_back(b[j]);
55          j++;
56      }
57  }
58–65  a lista que sobrou entra inteira, já em ordem
```



0 é o menor · 3 está nas duas e entra uma vez · 4 · 5 · a acabou, o resto de b (9) entra inteiro.

L46–49 iguais: entra uma vez e os dois andam, então nada se repete.

L50–55 entra o menor; o resultado sai crescente, pronto para o próximo filtro.

O(|a| + |b|) · Comedy OU Drama: 1,27 ms

Versão anterior: procurava cada item de b dentro de a inteira, 5.600 ms

MÓDULO 1 · RESULTADO MEDIDO

18.900 ms → 1,27 ms

tvEpisode E Drama: 474.707 × 191.271 posições cruzadas

Busca	Versão anterior	Sem índice	Agora
short E Animation	371 ms	6,2 ms	0,13 ms
movie E Comedy	737 ms	6,1 ms	0,25 ms
tvEpisode E Drama	18.900 ms	14,4 ms	1,27 ms
Comedy OU Drama	5.600 ms	—	1,27 ms

A versão anterior comparava cada item com a outra lista inteira, O(|a|·|b|): era mais lenta até que olhar a base sem índice. Agora é **14.900× mais rápida** nesse caso.

Fonte: tests/logs/benchmark.tsv · mediana de 3 execuções

MÓDULO 2 · PASSO 1: BALDES

Os valores ficam em ordem sem ordenar nada

src/arvore.cpp · adicionar() · linhas 3–10

```
3   void ArvoreNumerica::adicionar(int valor, int indiceFilme) {
4       if (valor < 0) {
5           return;
6       }
7       if (valor >= static_cast<int>(filmesPorValor.size())) {
8           filmesPorValor.resize(valor + 1);
9       }
10      filmesPorValor[valor].push_back(indiceFilme);
```

src/arvore.cpp · montar() · linhas 15–24

```
15  for (size_t valor = 0; valor < filmesPorValor.size(); valor++) {
16      if (!filmesPorValor[valor].empty()) {
17-23     cria o nó {valor, -1, -1} e faz nos.push_back(novo)
24  raiz = ligar(0, static_cast<int>(nos.size()) - 1);
```

```
chegam: 120, 45, 90, 45
[45]  → 1, 3
[90]  → 2
[120] → 0
nos = [45, 90, 120]
```

L7–10 o índice do vetor **é** o valor: o filme vai direto para o seu balde.

L15 o contador anda 0, 1, 2…: os nós saem em ordem sem comparar valores (ideia do counting sort).

MÓDULO 2 · PASSO 2: MONTAR PELO MEIO

ligar(): o meio vira o topo, recursivamente

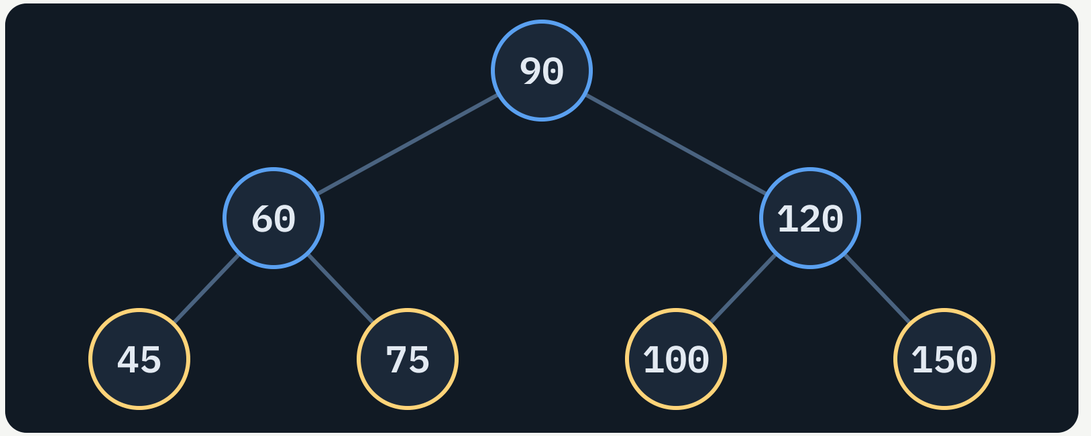
src/arvore.cpp · ligar() · linhas 27–35

```
27  int ArvoreNumerica::ligar(int inicio, int fim) {
28      if (inicio > fim) {
29          return -1;
30      }
31      int meio = (inicio + fim) / 2;
32      nos[meio].esquerda = ligar(inicio, meio - 1);
33      nos[meio].direita = ligar(meio + 1, fim);
34      return meio;
35  }
```

L28–29 inicio > fim = pedaço com **0 nós**. Cada chamada passa pedaços menores (7 → 3 → 1 → 0): a recursão sempre termina.

L31–33 o meio vira o topo; a metade menor vira a subárvore esquerda e a maior, a direita.

Folha: pedaço de 1 nó; as duas chamadas seguintes recebem pedaço vazio e devolvem −1.

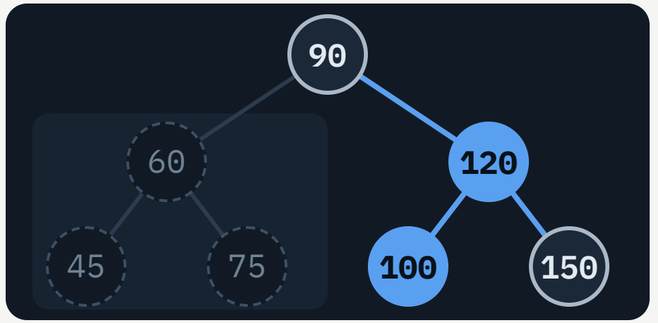



```
ligar(0,6) → meio 3 = 90
  ligar(0,2) → 60 · ligar(4,6) → 120
    ligar(0,0) → 45 · ligar(0,-1) → -1
```

coletar(): percurso em-ordem com poda

src/arvore.cpp · coletar() · linhas 46–61

```
46  void ArvoreNumerica::coletar(int no, int minimo, int maximo,
                                 std::vector<int>& resultado) const {
47      if (no < 0) {
48          return;
49      }
50      const No& atual = nos[no];
51      if (atual.valor > minimo) {
52          coletar(atual.esquerda, minimo, maximo, resultado);
53      }
54      if (atual.valor >= minimo && atual.valor <= maximo) {
55          const std::vector<int>& filmes = filmesPorValor[atual.valor];
56          resultado.insert(resultado.end(), filmes.begin(), filmes.end());
57      }
58      if (atual.valor < maximo) {
59          coletar(atual.direita, minimo, maximo, resultado);
60      }
61  }
```



Busca 95..130. L51 90 > 95? não: a esquerda inteira é pulada.

L54 100 e 120 estão no intervalo: o balde inteiro é copiado. A esquerda vem antes, então o resultado sai crescente.

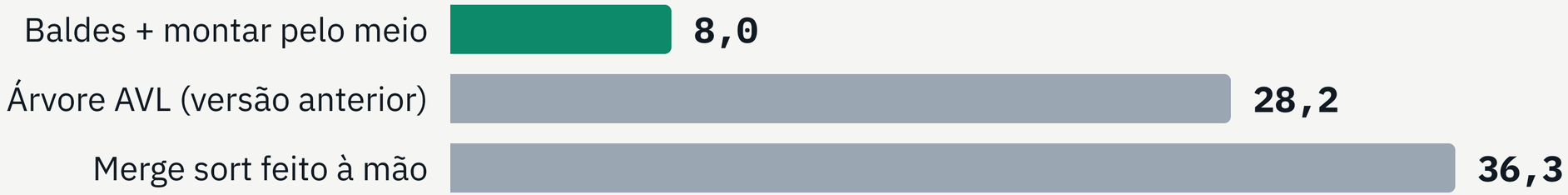
L58 150 < 130? não: não desce mais.

nós visitados + R cópias · ano 2000: 7 de 134

MÓDULO 2 · DECISÃO

Por que não AVL: medimos as três opções

Montar as estruturas de ano e de duração com os 750 mil valores, em ms



Depois de carregar, nenhum filme entra: **não há o que reequilibrar**. Montada pelo meio, a árvore fica ainda mais baixa que a AVL.

Fonte: tests/logs/benchmark.tsv · mediana de 3 execuções

Busca binária no carregamento: código → posição

src/carregador.cpp · resolverFilmesDosCinemas() · 114–127

```
114  int inicio = 0;
115  int fim = static_cast<int>(filmes.size()) - 1;
116  while (inicio <= fim) {
117      int meio = inicio + (fim - inicio) / 2;
118      if (filmes[meio].id == codigo) {
119          posicao = meio;
120          break;
121      }
122      if (filmes[meio].id < codigo) {
123          inicio = meio + 1;
124      } else {
125          fim = meio - 1;
126      }
127  }
```

```
cc00002 procura tt8000034
passo  1 · 584.121 candidatos
passo  2 · 292.060
passo  3 · 146.029
…
passo 19 · 2 → achou: posição 37.821
```

L117 inicio + (fim – inicio) / 2 dá o mesmo meio que (inicio + fim) / 2, sem risco de estourar o int.

L122–125 compara os códigos como texto e descarta a metade onde o código não pode estar.

Código inexistente: o laço termina com **inicio** no próximo código maior, exatamente o que o Módulo 4 pede.

CINEMAS · OPÇÕES 2 E 5 DO MENU

cinemasQueExibem: marcar os filmes e conferir cada cinema

src/main.cpp · cinemasQueExibem() · 146–160

```
146  std::vector<char> procurado(totalFilmes, 0);
147  for (int indice : indicesFilmes) {
148      procurado[indice] = 1;
149  }
151  std::vector<int> encontrados;
152  for (size_t i = 0; i < cinemas.size(); i++) {
153      for (int posicao : cinemas[i].filmePosicoes) {
154          if (posicao >= 0 && procurado[posicao]) {
155              encontrados.push_back(static_cast<int>(i));
156              break;
157          }
158      }
159  }
160  return encontrados;
```

```
procurado  0  1  0  0  1
 cinema A  [3, 4]  → procurado[4] = 1 → entra
 cinema B  [0, 2]  → nenhum → fora
 cinema C  [−1, 1] → entra
```

L146–148 um char por filme (584 KB): marca com 1 os filmes achados pela busca (índice ou árvore).

L154 conferir é acesso a vetor, não comparação de texto; −1 (código inexistente) é ignorado; o break para no primeiro achado.

TODA BUSCA · CRONÔMETRO E CACHE

buscarFilmes: mede só a busca e guarda o resultado

src/main.cpp · buscarFilmes() · linhas 193–204, sem as linhas em branco

```
193  std::string chave = montarChaveCache(filtros, operador, "filme");
195  auto inicio = std::chrono::steady_clock::now();
197  std::vector<int> resultado;
198  bool veioDoCache = cache.buscar(chave, resultado);
199  if (!veioDoCache) {
200      resultado = aplicarFiltros(filtros, operador, indiceTipo, indiceGenero);
201      cache.adicionar(chave, resultado);
202  }
204  double milissegundos = milissegundosDesde(inicio);
```

src/cache.cpp · CacheConsultas::buscar() · 4–8

```
4  for (size_t i = 0; i < chaves.size(); i++) {
5      if (chaves[i] == chave) {
6          resultado = resultados[i];
7          return true;
8      }
```

L195 / L204 o cronômetro só roda depois da digitação e para antes de imprimir.

L198–201 chave já vista: devolve o resultado guardado; senão calcula e guarda. tvEpisode E Drama: 1,27 ms → 0,02 ms.

DESEMPENHO · BUSCAS

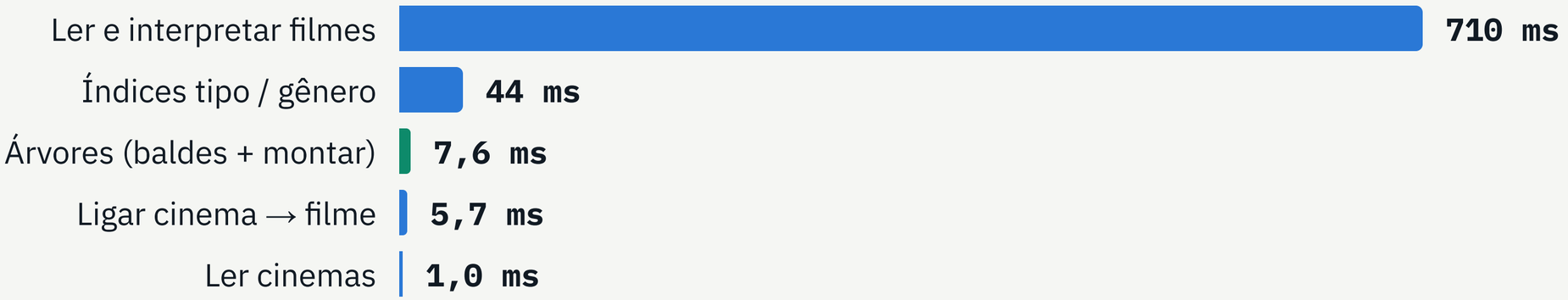
Árvore × olhar os 584 mil filmes

Busca	Achados	Nós visitados	Árvore	Sequencial	Vantagem
Ano 2000	5.192	7 de 134	0,0006 ms	3,67 ms	**6.411×**
Duração 90..120	8.501	39 de 372	0,0036 ms	2,57 ms	**711×**
Anos 1990..1999	57.239	16 de 134	0,025 ms	3,46 ms	**136×**
Duração 1..59	153.039	64 de 372	0,074 ms	4,48 ms	**61×**
Anos 1885..2026	559.303	134 de 134	0,98 ms	5,02 ms	**5,1×**

Fonte: tests/logs/benchmark.tsv · mediana de 3 execuções

DESEMPENHO · ABERTURA

Abrir leva 0,75 s: 92% é ler o arquivo



Fonte: tests/logs/benchmark.tsv · grupo "carregamento"

DE ONDE VÊM OS NÚMEROS

Todo número foi medido rodando código

1. O próprio programa
mede e mostra o tempo ao abrir (0,75 s) e em cada busca

2. O teste automático
703 buscas seguidas, cada tempo anotado: média de 0,03 ms

3. O benchmark
programa à parte que mede o nosso código e as alternativas que não usamos

tests/benchmark/benchmark.cpp · medir()

```
20  static double medir(Operacao operacao, int repeticoes) {
21      auto inicio = Relogio::now();
22      for (int i = 0; i < repeticoes; i++) {
23          operacao();
24      }
25      return milissegundosDesde(inicio) / repeticoes;
26  }
```